

POWERFACTORY

PowerFactory 2024

Technical Reference

Logic/DIP

RelLogdip, TypLogdip

PF2024

POWER SYSTEM SOLUTIONS

MADE IN GERMANY

Publisher:
DIgSILENT GmbH
Heinrich-Hertz-Straße 9
72810 Gomaringen / Germany
Tel.: +49 (0) 7072-9168-0
Fax: +49 (0) 7072-9168-88
info@digsilent.de

Please visit our homepage at:
https://www.digsilent.de

Copyright © 2024 DIgSILENT GmbH
All rights reserved. No part of this publication may be reproduced or distributed in any form without written permission of DIgSILENT GmbH.

January 9, 2024
PowerFactory 2024
Revision 1

Contents

1 General Description **1**

2 Logic/DIP Type (TypLogdip) **1**

2.1 Usage . . . 2

2.1.1 DIP switch . . . 2

2.1.2 Logic . . . 2

2.1.3 DIP and logic . . . 3

2.1.4 Switch logic . . . 3

2.1.5 Other entries . . . 3

2.2 Equations . . . 3

2.2.1 Analogue . . . 3

2.2.2 Digital . . . 4

2.2.3 References . . . 5

2.2.4 Special Functions . . . 5

3 Logic/DIP (RelLogdip) **6**

3.1 Basic Data . . . 6

3.2 Logic . . . 6

3.3 DIP Settings . . . 7

1 General Description

This document describes the configuration of *Logic/DIP* units for protection devices in *PowerFactory*.

User-defined logics can be used to model more advanced trip logic configurations, such as single-pole tripping or signal comparison schemes, or perform additional calculations with input signals.

2 Logic/DIP Type (TypLogdip)

The *Logic/DIP Type* defines basic capabilities and the default logic.

The general usage of the logic is specified with the *Used for* selection. Detailed explanations and limitations for each entry can be found in Section 2.1. The *User-defined settings* selection specifies whether logic and/or DIP settings can be modified in the element. The *Type* selection specifies which kind of equations are supported. Please refer to Section 2.2 for detailed explanations. The *Breaker event* field specifies potential breaker interactions. Breaker events can only be fired from digital logics.

For logics with DIP switches the *Default DIP settings* can be defined. The *Input signals*, *Output signals* and *Tripping signals* are used to specify input and output signals used in the logic. Only logics configured with a breaker event require the tripping signals. All three signal inputs expect the signals as comma-separated list without blanks.

The selections for *Reference block 1* and *Reference block 2* can be used as a way to access additional parameters from other units within relay. Please refer to Section 2.2 for additional information.

The actual default logic is entered on the *Logic* page.

2.1 Usage

2.1.1 DIP switch

DIP switches are a very simple form of logic which allow to *open* or *close* the connection.between an input and an output signal.

The number of input signals must be equal to the number of output signals. The actual logic will be created automatically. An example for three input signals `w1,w2,w3`, output signals `y1,y2,y3` and the DIP setting `101` is shown in Figure 2.1:

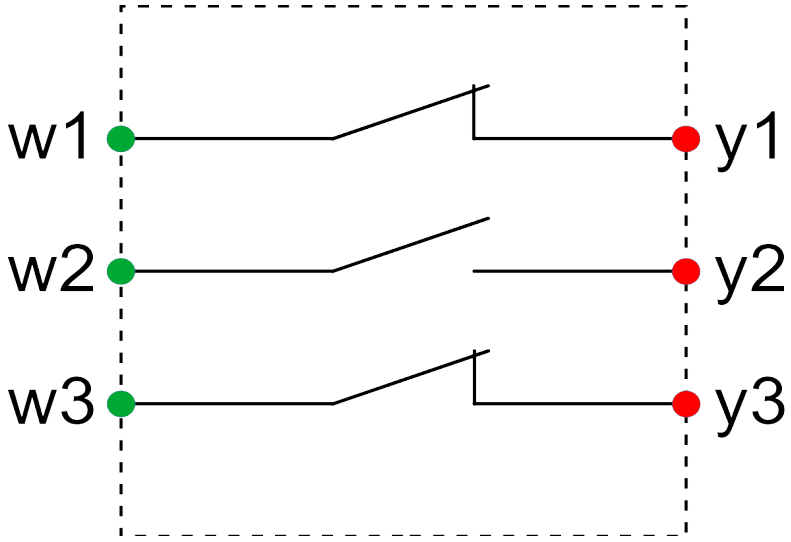


Figure 2.1: Example for DIP switches

Depending on the (default) DIP state the following values are possible for the output signals:

"off" (0) the DIP switch is open

"on" (1) the DIP switch is closed

"off (high)" (2) the DIP switch is open; the signal is *high* if the input is not connected

"on (high)" (3) the DIP switch is closed; the signal is *high* if the input is not connected

DIP switches can be used with analogue and digital signals, but the two DIP states "off (high)" and "on (high)" should only be used with digital logics and tripping signals, as other signals have no pre-defined value for *high* (see also Section 2.2).

2.1.2 Logic

Logic allows user-defined logic to be implemented. This can be used to modify input signals for other units, such as a restraint calculation for differential protection, or for advanced tripping time manipulation, such as distance protection signalling schemes. The logic is entered on the "Logic" page. Depending on the selected *Type* different restrictions and operators apply. Please refer to Section 2.2 for more information.

2.1.3 DIP and logic

Dip and logic is a combination of the two previous modes. The DIP switches are applied to the input signals which are then used in the logic.

2.1.4 Switch logic

Switch logic is a variant of the *Logic* which allows to access and process switch states. Up to 10 switches can be accessed via automatically generated signals `_SWT1` through `_SWT10`, where the number corresponds to the position in the switch table of the element (see Section 3). *Switch logic* can not be used as a tripping logic.

2.1.5 Other entries

The remaining entries *Dip+Latched switch*, *Setting group logic* and *DIP commutator* are deprecated and should not be used.

2.2 Equations

This section describes differences for the possible equations for analogue and digital logics. As a general rule, digital logics should be used when working with tripping signals and analogue units when working with physical quantities, such as currents or voltages.

2.2.1 Analogue

An *Analogue* logic is similar to a DSL equation and support the basic mathematical operations and the "DSL Standard Functions" (please refer to the *PowerFactory* User Manual). These types of logics are mostly used for adaption of input signals and can not be used to generate tripping events.

An example logic, calculating a restraint current, is shown below.

```
norm = (IrCT1 - IrCT2) * (IrCT1 - IrCT2) + (IiCT1 - IiCT2) * (IiCT1 - IiCT2)

Istab = sqrt(norm) / 2
```

Listing 1: Example for an *Analogue* logic

The signal definitions for this example would be:

Input signals `IrCT1,IiCT1,IrCT2,IiCT2`

Output signals `Istab`

The internal variable `norm` is defined automatically. Without going into much detail about what the connected signals are, the output signal `Istab` will be the result of the following equation:

$$I_{stab} = \frac{\sqrt{(I_{r,CT1} - I_{r,CT2})^2 + (I_{i,CT1} - I_{i,CT2})^2}}{2} = \left| \frac{\underline{I}_{CT1} - \underline{I}_{CT2}}{2} \right|$$

2.2.2 Digital

Digital logics are mainly used to process tripping signals and therefore use a different syntax, focusing on logical operations.

An example logic, evaluating the tripping signal of a simple distance device, is shown below.

```
PUTT = NOTRIP

yout_A = wZ1_A.or.wZ2_A
yout_B = wZ1_B.or.wZ2_B
yout_C = wZ1_C.or.wZ2_C

yout = yout_A.or.yout_B.yout_C.or.{PUTT.and.wRec.and.wZ1b}
```

Listing 2: Example for a *Digital* logic

The signal definitions for this example would be:

Input signals `wZ1_A,wZ1_B,wZ1_C,wZ2_A,wZ2_B,wZ2_C,wZ1b,wRec`

Output signals *(None)*

Tripping signals `yout,yout_A,yout_B,yout_C`

Again, without going into too much detail, the three phase-wise tripping signals (`yout_A`, `yout_B` and `yout_C`) will indicate a trip if the corresponding phase-wise input signal from zone 1 or 2 indicate a trip. The general tripping signal (`yout`) will indicate a trip, if either of the phase-wise signals indicates a trip or if the signal comparison is enabled and the corresponding remote (`wRec`) and local (`wZ1b`) signals indicate a trip. The signal comparison is disabled by default in this example, for an explanation on how to enable it for individual relays, please refer to Section 3.

The three main logical operations are shown in Table 2.1. The table also shows an alternative syntax for *and* and *or* which is only available in digital logics. Expressions can be grouped with { `expression` }, as shown with the signal comparison.

Note: Due to the different meaning of the `+` and `*` operators, digital logics can not be used effectively for calculation of input values.

The constants `TRIP` and `NOTRIP` are always available for digital logics. They can be used, as shown in the example, to enable or disable auxiliary variables. Internally they will be replaced with the appropriate value for dynamic and steady-state calculation.

Operation	Alternative Notation	Boolean Logic
`A.and.B`	`A*B`	`A && B`
`A.or.B`	`A+B`	`A || B`
`.not.A`		`!A`

Table 2.1: *Digital* logic operations

2.2.3 References

The two references *Reference block 1* (prefblock1) and *Reference block 2* (prefblock2) can be used to access setting values from other units within a relay, which are not available as a signal.

An example, calculating a per-unit current in an analogue logic, is shown below.

```
iabs = Iabs / _REF1Inom
```

Listing 3: Example using a reference parameter

The signal definitions for this example would be:

Input signals `Iabs,_REF1Inom`

Output signals `iabs`

If the *Reference block 1* is configured as a Measurement Type (*TypMeasure*) of the relay, the logic will replace `_REF1Inom` with the value of the *Nominal Current* (Inom) parameter of the measurement.

In general, each reference parameter has to be defined as an input signal with the following form: `_REF1` or `_REF2` followed by the parameter name. The number determines which reference selection is used. Only element parameters can be accesses, but it is possible to use multiple reference parameters from the same element (e.g. `_REF1Inom` and `_REF1Unom`).

Note: This mechanism is mostly used as a workaround for missing functionality. It gives very little indication for errors, such as misspelt parameters, and should be used with care. It is also recommended to be used with analogue logics.

2.2.4 Special Functions

For digital logics only, two special functions are available to conveniently process loop-encoded signals, which may be generated from starting or directional units. Please refer to the corresponding Technical References for more information about the loop-encoded signals.

containsLineGroundLoop(loop, phase) indicates whether the signal *loop* contains the loop for *phase* A (0), B(1) or C (2)

containsLineLineLoop(loop, phase) indicates whether the signal *loop* contains the loop for *phase* AB (0), BC(1) or CA (2)

The following example deconstructs the given loop signal into dedicated signals for each of the six possible loops for further processing.

```
y_A = containsLineGroundLoop(wloop, 0)
y_B = containsLineGroundLoop(wloop, 1)
y_C = containsLineGroundLoop(wloop, 2)

yl_AB = containsLineLineLoop(wloop, 0)
yl_BC = containsLineLineLoop(wloop, 1)
yl_CA = containsLineLineLoop(wloop, 2)
```

Listing 4: Example loop signal functions

3 Logic/DIP (RelLogdip)

The *Logic/DIP* unit is used to further configure the logic for the individual relay where required.

3.1 Basic Data

If the logic is configured with breaker interactions or as a *Switch Logic*, a table for circuit-breakers is available on the Basic Data page.

By default only the first row of the table is available and empty. An empty field in the first row will prompt the logic to determine the breaker automatically, based on the location of the relay. If additional breakers shall be tripped (e.g. for transformer differential protection), additional rows can be added to the table. Please note, that additional breakers must be explicitly defined and are not determined automatically.

If the logic has more than one *Trip Signal* defined, a selection for the tripping signal(s) is available on the Basic Data page. If any of the selected signals indicates a trip, the logic will be considered as tripped. In steady-state calculations this will set the corresponding *Tripping Time* parameter of the relay. In dynamic simulation a tripped logic will generate a switch event for each of the breakers in the table.

The *Trip Signal* field can also be manually edited to defined a more than one, but less than all, signals as trip signals. As with other signal list, the chosen trip signals should be entered as a comma-separated list without blanks.

The *Logic/DIP* unit also supports single-pole tripping if also supported by the calculation. if the trip signal ends on `_A`, `_B` or `_C`, only the respective phase will be considered as tripped.

3.2 Logic

The *Logic* page is available if the logic is configured as user-definable in the type. A user-defined logic can be used modify existing equations. Signal assignments in the user-defined logic will replace the corresponding assignment in the default logic.

For example, in order to activate the signal transmission from the *Digital* logic example, the user-defined logic would be as follows.

```
PUTT = TRIP
```

Listing 5: Example for user-defined logic

The logic from Listing 5 will be combined with the default logic from Listing 2, resulting in the logic shown below.

```
PUTT = TRIP

yout_A = wZ1_A.or.wZ2_A
yout_B = wZ1_B.or.wZ2_B
yout_C = wZ1_C.or.wZ2_C

yout = yout_A.or.yout_B.yout_C.or.{PUTT.and.wRec.and.wZ1b}
```

Listing 6: Final logic combining user-defined and default logic

Alternatively, the user-defined logic can also be used to change the way the tripping signal is generated.

```
yout = yout_A.or.yout_B.yout_C
```

Listing 7: Alternative example for user-defined logic

Combining Listing 7 with Listing 2 would result in a logic without signal transmission, as shown below.

```
PUTT = NOTRIP

yout_A = wZ1_A.or.wZ2_A
yout_B = wZ1_B.or.wZ2_B
yout_C = wZ1_C.or.wZ2_C

yout = yout_A.or.yout_B.yout_C
```

Listing 8: Final logic using the alternative user-defined logic

3.3 DIP Settings

The *DIP Settings* page is available if the DIP switches are configured as user-definable in the type. It allows to select the desired state for each of the DIP switches.